Lecture 2: Validation, Overfitting, and Forecast Failure

Ivy Yang[1]

[1]Economics & Finance, Business School, Southern Illinois University - Edwardsville

Overfitting and underfitting (intuition-based)

Definition: Overfitting / Underfitting

Overfitting means the model learns not only signal but also accidental noise in the training period, so performance drops on new data. Underfitting means the model is too simple to capture the stable pattern in the first place.

In practice, choosing model complexity is a bias–variance balancing problem, not a contest for the smallest in-sample error. A simpler model is usually more stable but may miss persistent structure, while a flexible model can capture richer patterns at the cost of higher sensitivity to noise. In high-stakes settings such as trading, credit approval, or inventory allocation, practitioners are typically more concerned about overfitting because false confidence can propagate directly into costly decisions. A familiar example is a strategy that looks excellent after repeated backtest tuning but degrades quickly once deployed.

Example 2.1. *In the notebook, AR(p) is estimated for $p = 1, \ldots, 20$ on S&P 500 log returns. As p increases, training RMSE usually keeps falling, but test RMSE improves only up to a point and then rises. This is the practical sign that complexity is starting to hurt generalization.*

Example 2.2. *A simple way to track this is to compare*

$$RMSE_{train} = \sqrt{\frac{1}{n_{tr}} \sum (y_t - \hat{y}_t)^2}, \qquad RMSE_{test} = \sqrt{\frac{1}{n_{te}} \sum (y_t - \hat{y}_t)^2}.$$

When $RMSE_{test} - RMSE_{train}$ stays positive and grows with model complexity, we should treat it as an overfitting warning.

Validation for time-indexed data

Definition: Time-aware validation

Time-aware validation enforces one rule: at time t, the model can only use information that was actually available by time t.

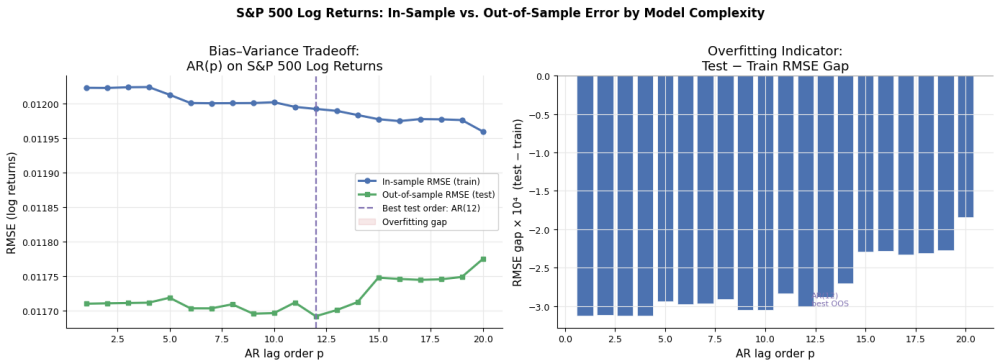

Figure 1: Example 2.1 notebook output: AR(p) train/test RMSE and overfitting gap (cell 9).

Forecast evaluation is only informative when it mirrors the information set available at the time of decision. Chronological validation sacrifices some convenience relative to random splitting, but it preserves causal interpretation and external validity. For sequential decisions in finance, policy, and operations, the main risk is not a slightly noisier estimate—it is a systematically optimistic estimate caused by unrealistic data access. For instance, a month-ahead demand model should not rely on variables that are finalized after the forecast cutoff date.

Example 2.3. *The notebook uses a chronological 60%/20%/20% split on S&P 500 levels. The train segment is used to estimate candidate models, the validation segment is used for model choice, and the test segment is kept untouched for the final unbiased report.*

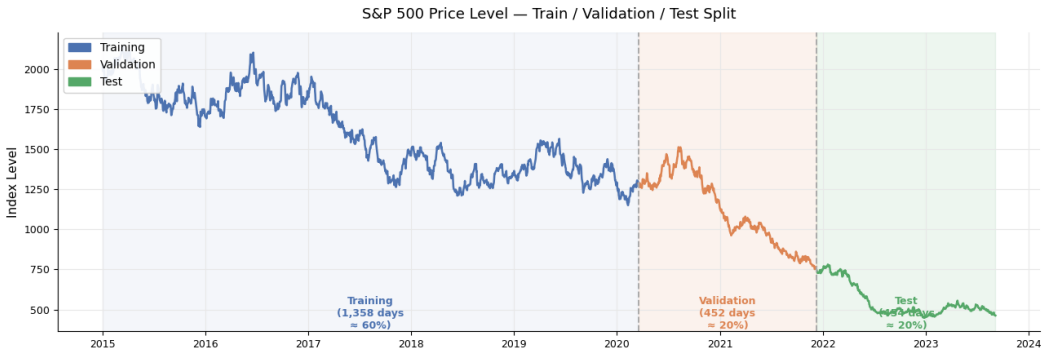

Figure 2: Example 2.3 notebook output: chronological train/validation/test split (cell 6).

Data leakage demonstration

Definition: Data leakage

Data leakage happens when future information accidentally enters the training process, making evaluation look stronger than what is possible in real forecasting.

Data leakage should be viewed as an identification problem in the modeling pipeline, not merely a coding mistake. Richer features can improve apparent fit, but that gain is meaningless if any

transformation imports future information into training. In applications with asymmetric downside risk, a modestly misspecified model is often less dangerous than a leaky model because leakage creates structural overconfidence. A common case is using revised macroeconomic series instead of real-time vintages that were actually available when forecasts were made.

Example 2.4. *In notebook cell 12, the "leaky" specification uses y_{t+1} to predict y_t. That variable would not be observable at forecast time, so the result is not deployable. The fit may look excellent, but it is a false signal caused by timing violation.*

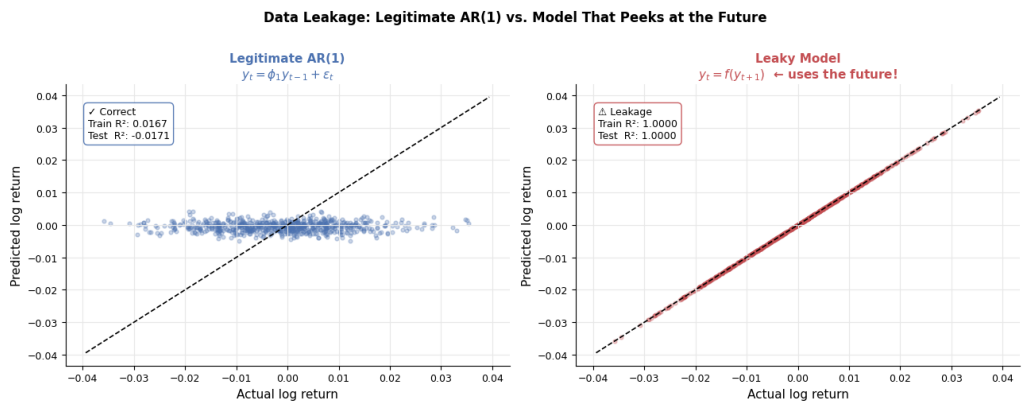

Figure 3: Example 2.4 notebook output: legitimate AR(1) vs. leaky specification (cell 12).

Rolling and expanding window validation

Definition: Rolling vs. expanding windows

Rolling window keeps a fixed-length recent history, while expanding window grows the training history over time. Both are repeated out-of-sample validation schemes.

Window choice determines how much historical memory the model carries into each forecast round. Rolling windows prioritize adaptation and are often useful under regime shifts, whereas expanding windows prioritize estimation stability by accumulating data over time. The tradeoff is therefore responsiveness versus noise robustness, and the preferred choice depends on how quickly the underlying process drifts. In promotion-heavy retail series, for example, rolling windows often adapt faster after campaign shocks than long expanding samples.

Example 2.5. *Rolling windows usually react faster to structural change because they emphasize recent data, but they can be noisier when the window is short. Expanding windows are typically more stable because they use more observations, but they may adapt more slowly to new regimes.*

Example 2.6. *In the notebook, rolling AR(1) uses roughly 120 trading days, while expanding AR(1) starts from a 50% burn-in sample and keeps adding new data. The comparison shows that neither method dominates in every period; method choice depends on how stable the environment is.*

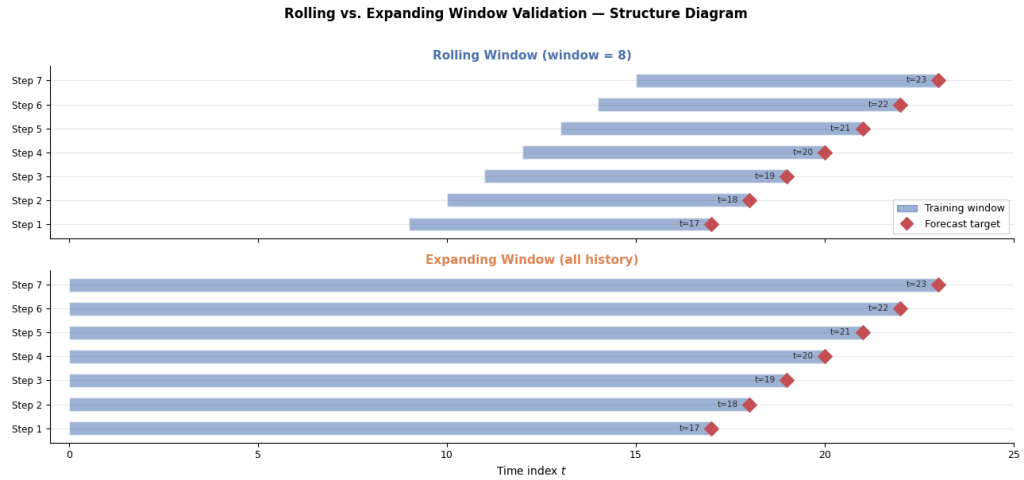

Figure 4: Example 2.6 notebook diagram: rolling versus expanding window structure (cell 14).

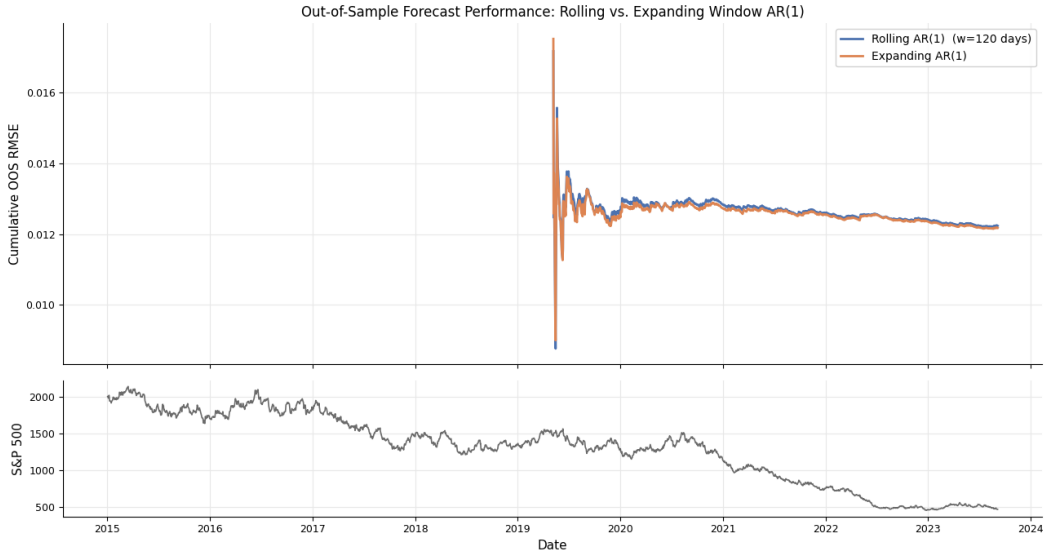

Figure 5: Example 2.6 notebook comparison: rolling vs. expanding AR(1) OOS RMSE (cell 15).

K-fold cross-validation with time-series caveats

Definition: Time-series cross-validation

Cross-validation means repeated skepticism across multiple splits. In forecasting, each validation fold must happen after its training fold in calendar time.

Cross-validation remains valuable for model selection in time series, but the fold design must respect temporal ordering. Increasing the number of folds can reduce evaluation roughness, yet this benefit is meaningful only when each validation block is genuinely out-of-time. In forecasting work, the central concern is usually temporal causality rather than minor variance differences

across CV schemes, because causality violations can reverse model rankings. A simple sanity check is that no model should be trained on later dates and validated on earlier ones.

Example 2.7. *Standard random K-fold assumes observations are exchangeable. Time-series observations are ordered, so random shuffling can quietly move future information into the training set and create optimistic bias.*

Example 2.8. *TimeSeriesSplit-style CV keeps ordering and better matches deployment logic. In the notebook, comparing $K = 5$, $K = 10$, and $K = 100$ illustrates a practical tradeoff: smoother average curves versus higher computational burden and possible sensitivity to fold design.*

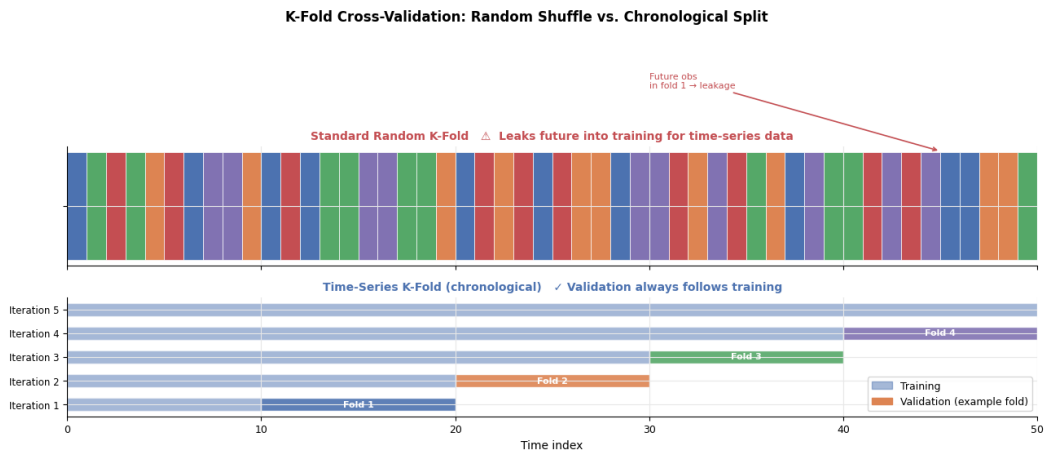

Figure 6: Example 2.8 notebook diagram: standard K-fold vs. time-series K-fold (cell 18).

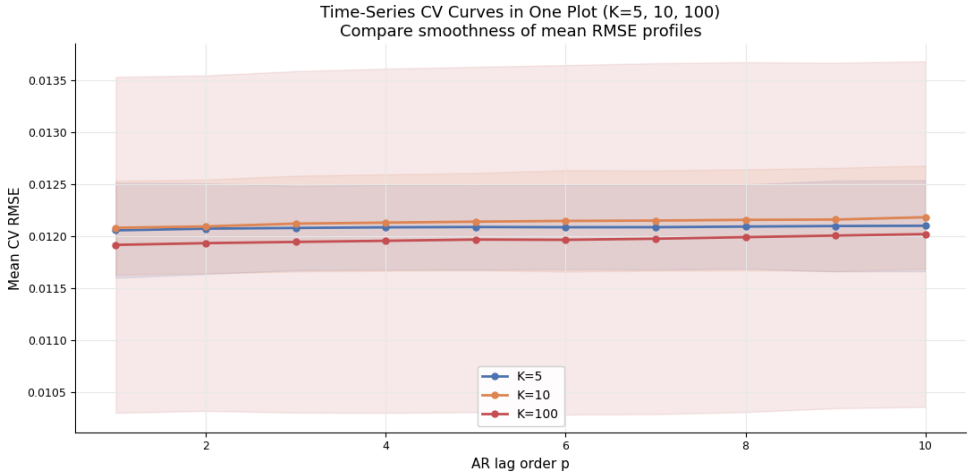

Figure 7: Example 2.8 notebook comparison across fold counts (cell 21).

Bootstrapping for forecast uncertainty

Definition: Bootstrap / Block bootstrap

NIST/SEMATECH defines bootstrapping as estimating the uncertainty of a statistic by repeatedly drawing subsamples with replacement from observed data, recomputing the statistic each time, and using the resulting values to approximate its sampling distribution. Block bootstrap is the time-series adaptation that resamples contiguous blocks so local dependence is preserved.

Definition adapted from: NIST/SEMATECH e-Handbook, Section 1.3.3.4.

Point forecasts alone are insufficient when decisions are sensitive to tail outcomes. Block bootstrap provides interval estimates by resampling contiguous observations, so short-run dependence is retained rather than artificially broken. The practical tradeoff is block length: longer blocks preserve serial structure better, while shorter blocks generate more resampling variation. In risk control and capacity planning, decision-makers typically prefer slightly wider but credible intervals over narrow intervals that understate uncertainty, such as in safety-stock design.

Example 2.9. *The notebook uses AR(2) with $B = 500$ bootstrap replications, block length $l = 20$, and horizon $H = 20$. The output includes point forecasts and quantile bands such as 50% and 80% intervals. As horizon increases, the fan widens, which reflects compounding uncertainty rather than model failure.*

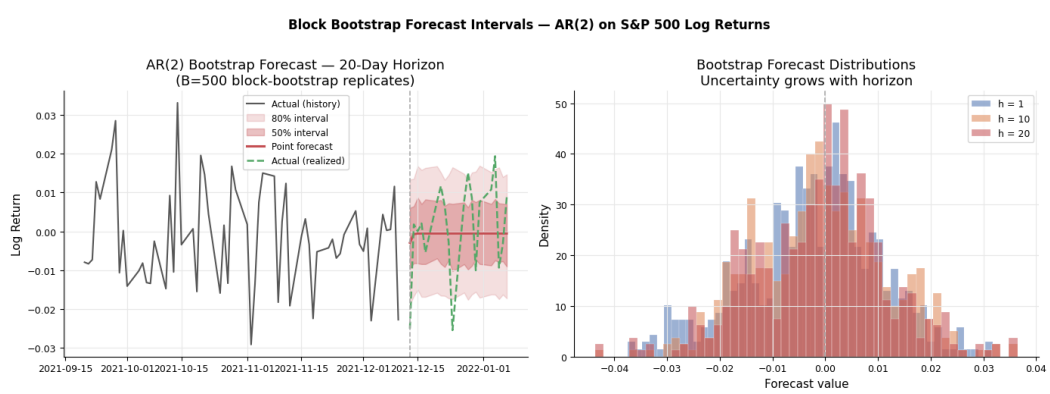

Figure 8: Example 2.9 notebook output: block-bootstrap forecast intervals (cell 23).

Why forecasts fail

Definition: Forecast failure

Forecast failure appears when model assumptions stop matching the environment, for example because of structural breaks, parameter drift, extrapolation beyond historical support, or data-process distortion.

Forecast failure is best interpreted as a mismatch between model assumptions and a changing data-generating process. Monitoring therefore requires balancing update frequency and operational stability: frequent updates adapt quickly but can add process noise, whereas slower updates are easier to govern but more exposed to regime miss. In turbulent environments, missing a structural break is often more harmful than tolerating temporary error volatility. Travel demand around pandemic disruptions is a clear example where pre-shock seasonal patterns lose explanatory power.

Example 2.10. *Regime change is the first common failure mode. The notebook trains on a pre-shock period and evaluates after a shock boundary (e.g., COVID), and rolling RMSE jumps sharply once the old regime no longer represents the new environment.*

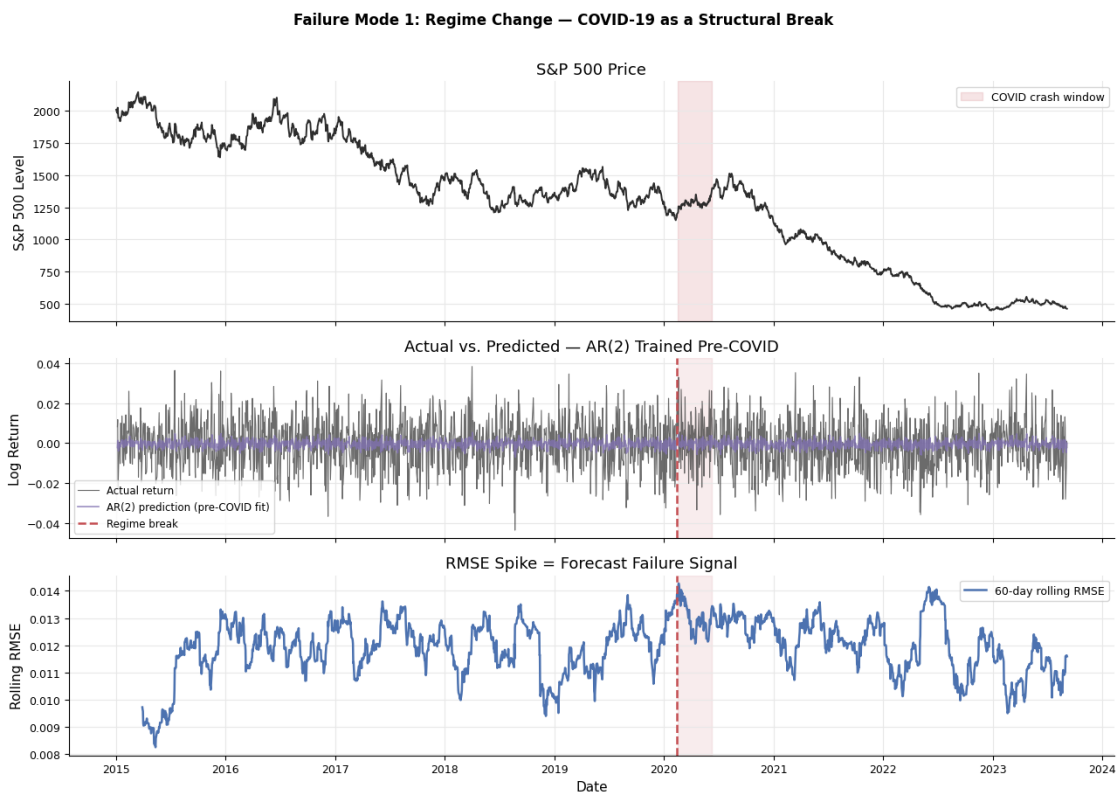

Figure 9: Example 2.10 notebook output: regime-change stress (cell 26).

Example 2.11. *Parameter instability is the second mode. Rolling AR(1) coefficient estimates drift over time, so a coefficient that looked reliable in one period can become weak or even misleading later.*

Example 2.12. *Extrapolation risk is the third mode. When forecasting inputs move outside the historical support, linear trend projection can produce fragile long-horizon predictions with low practical reliability.*

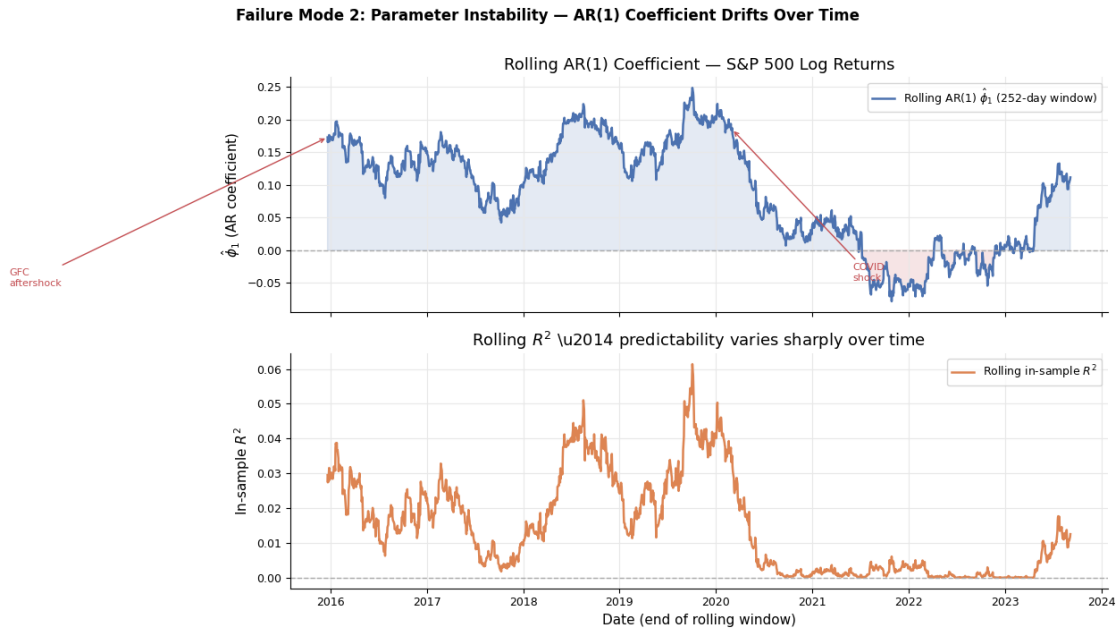

Figure 10: Example 2.11 notebook output: parameter drift in rolling estimates (cell 27).

Connection to later lectures

Definition: Stability-first logic

Lecture 2 builds the validation and failure-diagnosis framework needed before we discuss regularization, nonlinear methods, and model governance.

This chapter motivates a stability-first view of forecasting: model quality is judged by robustness under shift, not only by fit on a single sample. The key tradeoff is short-run accuracy gains versus long-run reliability and interpretability in governance settings. In communication-intensive decisions, slightly lower accuracy is often acceptable if uncertainty boundaries remain transparent and auditable. For executive reporting, a stable and interpretable model is usually more valuable than a marginally better but opaque and unstable alternative.

Example 2.13. *In Lecture 3, regularization becomes meaningful only after we can measure overfitting and instability clearly. In Lecture 4, nonlinear models require stricter leakage control and stronger validation design because flexibility increases failure risk.*

Example 2.14. *In Lectures 5–6, decision communication depends on transparent error boundaries, interval interpretation, and explicit failure conditions. In other words, model quality is judged by robustness and governance, not by in-sample fit alone.*

Figure 11: Example 2.12 notebook output: extrapolation beyond sample support (cell 28).